

Sistema de Feature Flags por Requisito

Informe de implementación — Estudio Larrañaga

Fecha: **03/06/2026 23:34**

Rama: **feature/req-flags-fase-1**

Commit: **d6242d1** (sistema en web) + **796370a** (PDF Fase 1)

Este documento describe el sistema de feature flags implementado para controlar la visibilidad de cada requisito R-XX en la plataforma. Incluye arquitectura, capturas de las vistas relevantes y una sección comparativa entre los permisos de super_admin y colaborador.

Contenido

1. Resumen ejecutivo
2. Arquitectura — Backend
3. Arquitectura — Frontend
4. Permisos y roles
5. Vistas
 - 5.1. Página /gestion-requisitos (super_admin)
 - 5.2. Dashboard super_admin con Modo verificación
 - 5.3. Dashboard colaborador
6. Diferencias super_admin vs colaborador (sección comparativa)
7. Cómo usarlo

1. Resumen ejecutivo

Cada uno de los 20 requisitos del Plan Maestro (R-01 a R-20) ahora tiene un toggle **enabled** que controla su visibilidad para los colaboradores. Solo los usuarios con rol **super_admin** pueden modificar estos toggles desde la página dedicada **/gestion-requisitos**. Los **admin** regulares y los **colaborador** no ven esa página ni pueden tocar los flags.

Funcionalidades:

- Toggle on/off por requisito con persistencia en base de datos.
- Modal de confirmación si se intenta activar un requisito no implementado.
- Sidebar dinámico: los items se ocultan automáticamente para colaboradores cuando su requisito está OFF.
- Rutas protegidas: URL directa a un requisito OFF redirige a /dashboard.
- Modo verificación opcional (solo super_admin): badges R-XX inline sobre cada item del sidebar para identificar visualmente qué corresponde a qué.
- Auditoría: cada toggle queda registrado en la tabla action_logs con usuario, fecha y valor.

2. Arquitectura — Backend

Modelo nuevo en **backend/app/models.py**:

Clase **FeatureFlag** con campos: codigo (PK), titulo, descripcion, area, fase, dificultad, ruta_frontend, implementado, enabled, updated_by_id, updated_at.

Schemas en **backend/app/schemas.py**:

- **FeatureFlagOut** — payload de lectura (todos los campos del modelo).
- **FeatureFlagUpdate** — solo **{enabled: bool}**.

Router en **backend/app/routers/feature_flags.py**:

- **GET /feature-flags** — cualquier usuario autenticado. Devuelve los 20 flags ordenados por fase y código.
- **PUT /feature-flags/{codigo}** — solo **super_admin** (depende de **require_super_admin** en **backend/app/routers/auth.py**). Cambia enabled, setea updated_by_id y updated_at, escribe entry en ActionLog.

Seed en **backend/app/mock_data.py**:

Función **seed_feature_flags()** con el catálogo de 20 R-XX (datos del Plan Maestro). Idempotente: crea los faltantes y refresca metadata sin tocar el campo enabled (eso lo decide el super_admin). Se ejecuta automáticamente en startup.

3. Arquitectura — Frontend

Context en **frontend/src/context/FeatureFlagsContext.jsx**:

FeatureFlagsProvider hace GET /feature-flags al login y mantiene en memoria los 20 flags. Expone los hooks **useFeatureFlag(codigo)** y **useFeatureFlags()**, más **setFlag(codigo, enabled)** que hace el PUT y actualiza el estado optimísticamente. También maneja el toggle **verificationMode** con persistencia en localStorage.

Componentes nuevos:

- **<FeatureGate codigo="R-XX">** — wrapper: si el usuario es admin siempre renderiza children; si no, solo si el flag está enabled. Acepta array de códigos (cualquier enabled basta).
- **<RequirementBadge>** — chip pequeño con color por estado (verde activo, amarillo inactivo, gris pendiente).

Página nueva — **/gestion-requisitos** (frontend/src/pages/GestionRequisitos.jsx):

- Solo super_admin (resto redirige a /dashboard).
- 4 tabs por fase, abierto Fase 1 por defecto.
- Cada R-XX es una card con código, título, descripción, área, dificultad, indicador implementado/pendiente y toggle on/off.
- Modal de confirmación cuando se intenta activar un requisito con implementado=false.
- Botón 'Modo verificación' en el header.

Sidebar modificado:

- Cada NavLink de la sección Acciones está envuelto en **<FeatureGate>**.
- Sección nueva 'Administración' al pie (solo super_admin) con el ítem 'Gestión de Requisitos' y el toggle 'Modo verificación'.
- Cuando Modo verificación está ON (solo super_admin), cada ítem muestra su badge R-XX a la derecha con el color según estado.

App.jsx — rutas gateadas:

13 rutas protegidas con **<Gated codigo="R-XX">** que redirigen a /dashboard si el requisito está OFF y el usuario no es admin: /herramientas, /honorarios, /profesionales, /liquidaciones, /retenciones, /cuentas-corrientes, /iva, /posicion-iva, /cobros, /maestro-proveedores, /flujo-fondos, /retiros, /actualizar-honorarios, /conciliacion-bancaria.

4. Permisos y roles

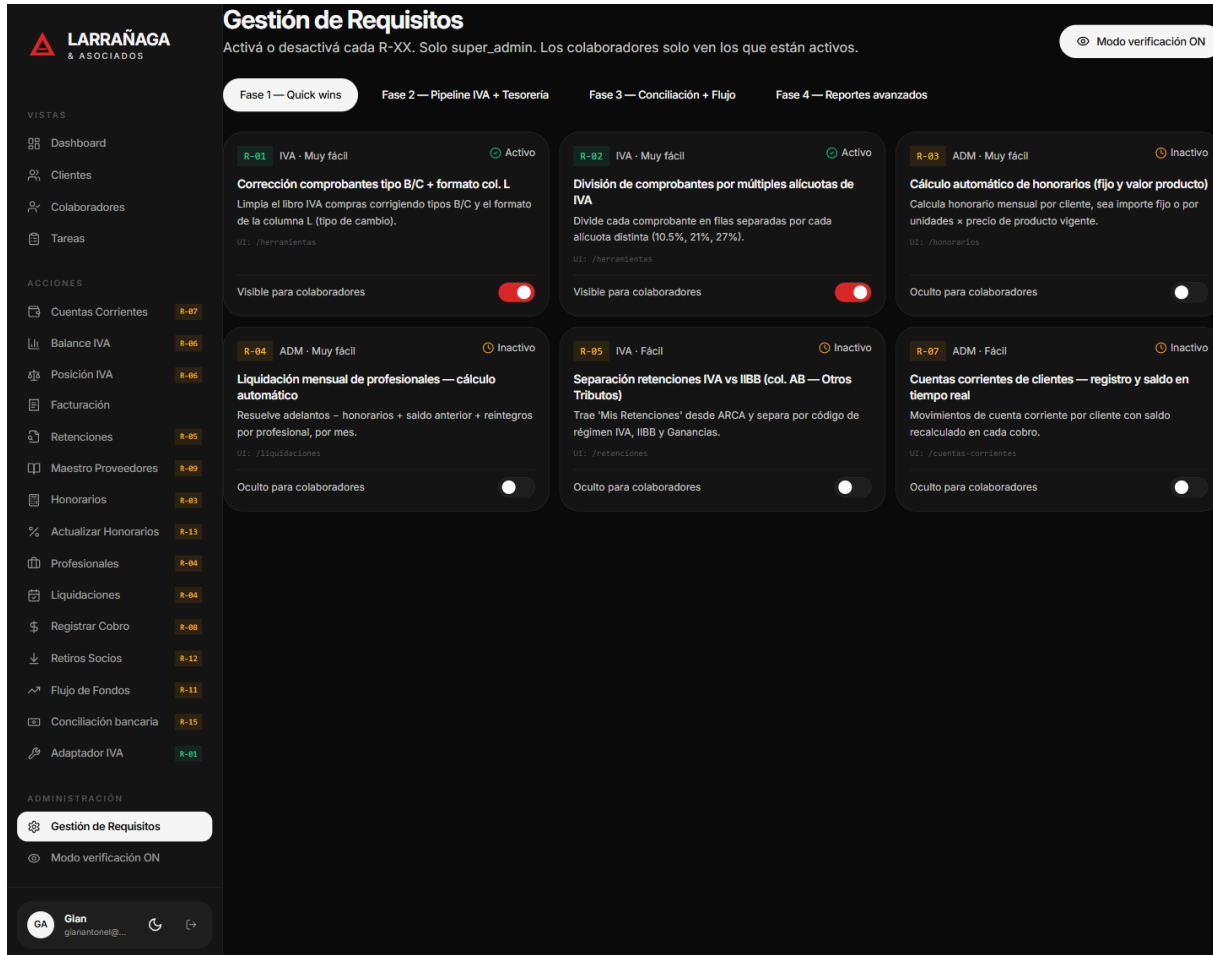
El enum UserRole del proyecto tiene cuatro valores: **super_admin**, **admin**, **colaborador**, **invitado**. La matriz de permisos del sistema de feature flags es:

Acción	super_admin	admin	colaborador	invitado
Ver lista de flags (GET)	Sí	Sí	Sí	Sí
Togglear un flag (PUT)	Sí	403	403	403
Acceder a /gestion-requisitos	Sí	→ /dashboard	→ /dashboard	→ /dashboard
Toggle 'Modo verificación' visible	Sí	No	No	No
Badges R-XX en sidebar	Sí (si modo ON)	No	No	No
Items del sidebar con req OFF	Visibles	Visibles	Ocultos	Ocultos
URL directa a req OFF	Renderiza	Renderiza	→ /dashboard	→ /dashboard

5. Vistas

5.1. Página /gestion-requisitos (super_admin)

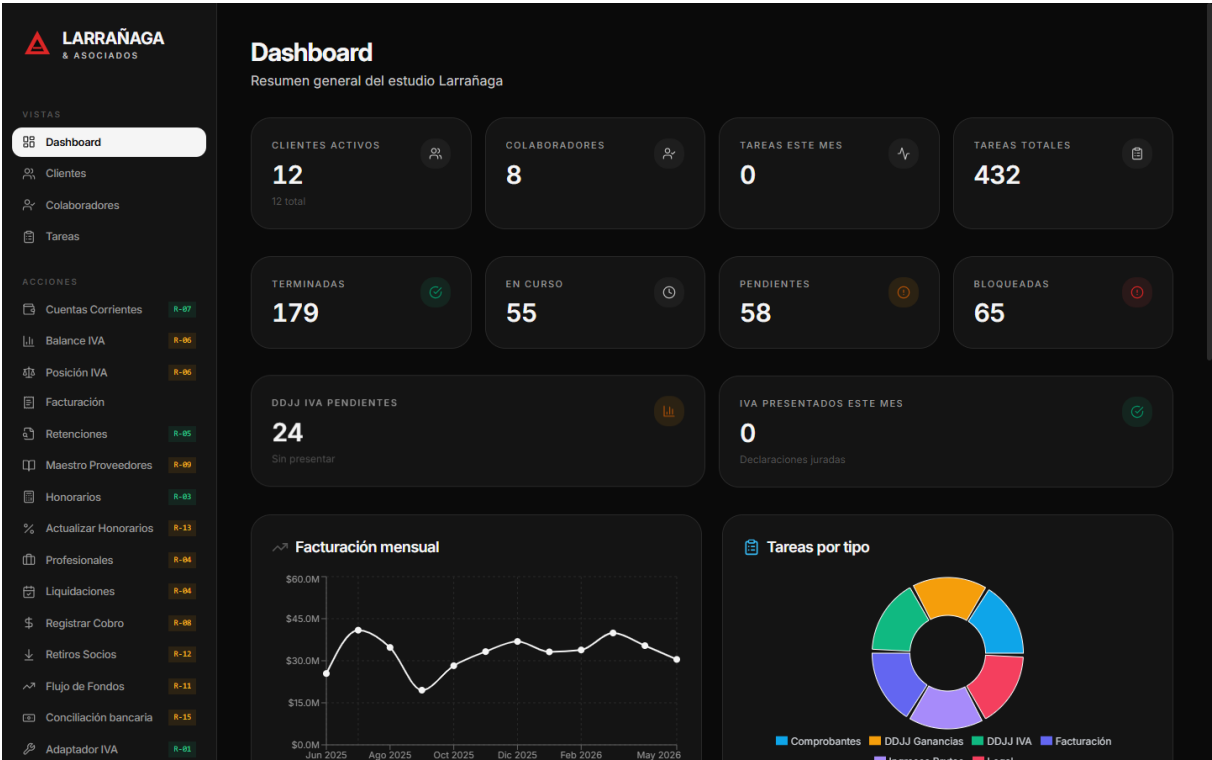
Página principal del sistema. Solo accesible para super_admin. Cuatro tabs (uno por fase), cards con toggle individual y modal de confirmación al activar pendientes.



Vista 1 — /gestion-requisitos en Fase 1, R-01 y R-02 activados.

5.2. Dashboard super_admin con Modo verificación

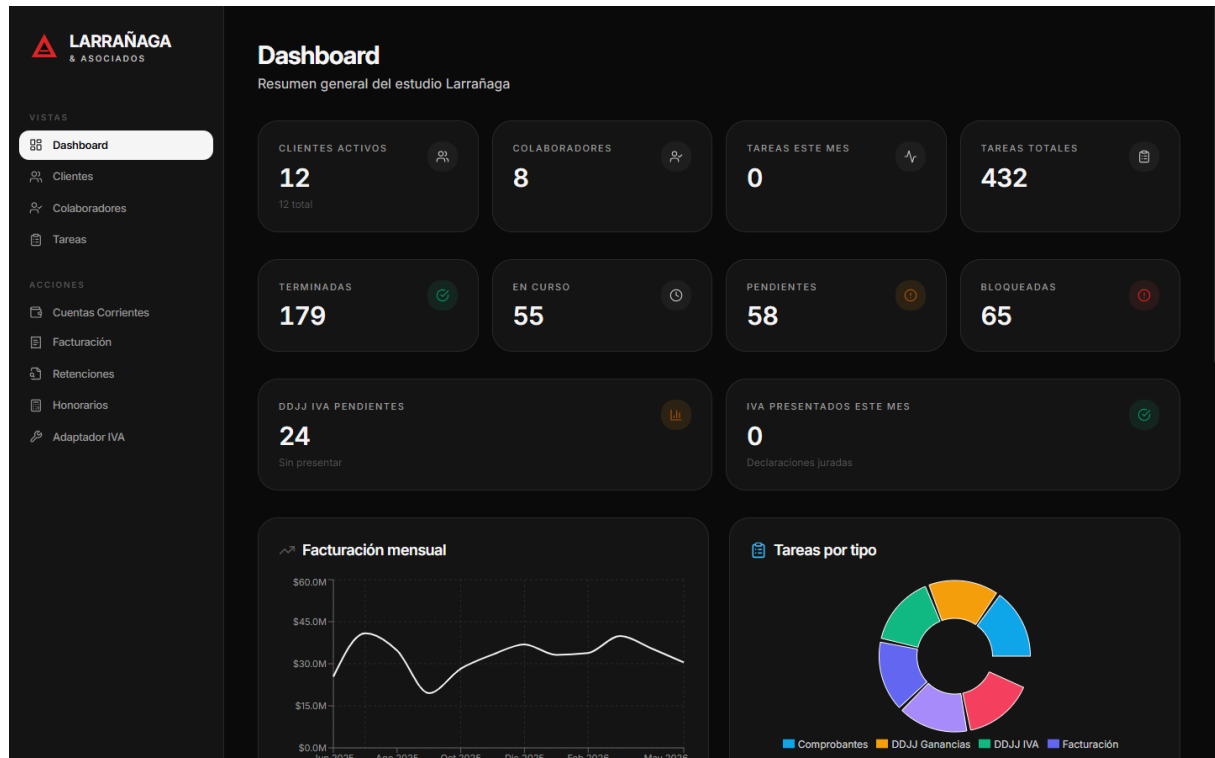
El super_admin ve TODOS los items del sidebar (incluso los que están OFF), más la sección Administración. Con el modo verificación activado, cada item muestra su badge R-XX a la derecha — verde si está activo, amarillo si está inactivo, gris si está pendiente.



Vista 2 — Dashboard como super_admin con Modo verificación ON.

5.3. Dashboard colaborador

El colaborador (rol colaborador) ve un sidebar reducido: solo los items cuyo requisito está enabled, más las vistas base y Facturación (que no tiene flag). No ve la sección Administración, no ve badges, no puede entrar a /gestion-requisitos.



Vista 3 — Dashboard como colaborador (mgonzalez@larranaga.com).

6. Diferencias super_admin vs colaborador

Esta sección compara directamente las Vistas 2 y 3 (mismas pantallas, distinto rol). Estado de los flags al momento de la captura: **R-01, R-02, R-03, R-05, R-07 activos**; el resto inactivo.

Aspecto	Super_admin (Vista 2)	Colaborador (Vista 3)
Items en sección Vistas	4 (Dashboard, Clientes, Colaboradores, Tareas)	Idénticos
Items en sección Acciones	15 (todos visibles, incluso los OFF)	5 (solo los que están enabled + Facturación)
Items ocultos para el colaborador	—	Balance IVA, Posición IVA, Maestro Proveedores, Actualizar
Sección Administración	Sí (Gestión de Requisitos + Modo verificación)	No existe
Badges R-XX inline	Sí (con Modo verificación ON): verde activo, amarillo inactivo, gris pendiente	No existe, no se puede ver el estado del Modo verificación
Acceso a /gestion-requisitos	Renderiza la página completa	Redirige inmediatamente a /dashboard
URL directa a un requisito OFF	Renderiza igual (admin ve todo)	Redirige a /dashboard (FeatureGate cierra)
Acción de togglear un flag	Permitido (200 OK)	403 Forbidden desde backend
Datos visibles en Dashboard	Idénticos (mismas tarjetas, mismas métricas, mismos gráficos)	Idénticos (flag solo afecta sidebar y rutas, no contenido de

Conclusión: el sistema es transparente para el contenido (mismo Dashboard, mismas métricas, mismos clientes). El gating actúa exclusivamente sobre *qué puede acceder* el usuario: items del sidebar y rutas. Eso permite ocultar features no validadas para el cliente sin tocar la lógica de negocio.

7. Cómo usarlo

Credenciales de prueba:

- **super_admin**: gianantonel@gmail.com / admin123 (también gerogambuli2002@gmail.com, rodriguezfederico765@gmail.com con admin123; optimizar.ai@gmail.com con optimizar123).
- **colaborador**: mgonzalez@larranaga.com / colab123 (entre otros 8).

Flujo típico para una demo al cliente:

1. Login como super_admin (Gian o Gero).
2. Entrar a sidebar → Administración → Gestión de Requisitos.
3. Activar solo los R-XX que se quieren mostrar al cliente (por ejemplo los 6 de Fase 1).
4. Logout y login como colaborador para verificar que solo ve esos items.
5. Cerrar la demo: super_admin desactiva los flags si quiere volver a ocultar las features.

Regenerar este informe:

```
cd backend && .venv/Scripts/python.exe scripts/generate_informe_pdf.py
```